Black Box Testing (A&O Ch. 4)

(2nd Ed. Ch. 6)

Course Software Testing & Verification

2025/26

Wishnu Prasetya & Paige North

Plan

- Partitioning based testing (A&O Ch 4/2nd Ed Ch. 6).
- Model based testing

Note: black box testing is an important concept. For example, system level testing is often done in a black box setup. In this lecture we will discuss two important techniques that are commonly used in such a setup. The first one, partition based testing, is discussed in length in A&O. However, the second one, model based testing (MBT), is only lightly touched in A&O. This lecture will introduce you to some basic concepts of MBT. For a more practical exposition on MBT you can look at e.g. Practical Model-Based Testing by Utting - Lageard (UU students can access it for free from Science Direct, https://www.sciencedirect.com/science/book/9780123725011)

White box testing

Testing is "**white box testing**" if you have knowledge of the source code of the target program to help you designing the tests, and your tests have in principle access to all the program's variables so that they can inspect them if they are in the correct state.

- Compare this with Def 1.26 A&O (2nd Ed. See p 26).
- White box setup is common at the unit-testing level.

Black box testing

Testing is ”**black box testing**” if you **do not** assume full knowledge of the inner working of the target program. Usually this also entails that your tests have only limited access to the program state to inspect its correctness.

- Compare with Def 1.25 in A&O (2nd Ed. See p26).
- Note that although we may have full access to the source code, at the system level we may choose not to use this knowledge because it becomes to complicated to comprehend.

Black box testing is common at the system-testing level. We will discuss two approaches : (1) **Partition-based testing**, Ch 4 (2nd ed. Ch. 6) and (2) **Model-based testing**.

Partition-based testing

- Premise:

The input space of a program can be partitioned into "equivalence classes", such that inputs from the same partition/equivalence-class lead to the "same kind" of behavior.

- It then makes sense to require that every partition should be tested at least once.
- Even without source code, we can often **propose** a reasonable partitioning.

Recall this example…

Enum TriType { Isosceles, Equilateral, Scalene }

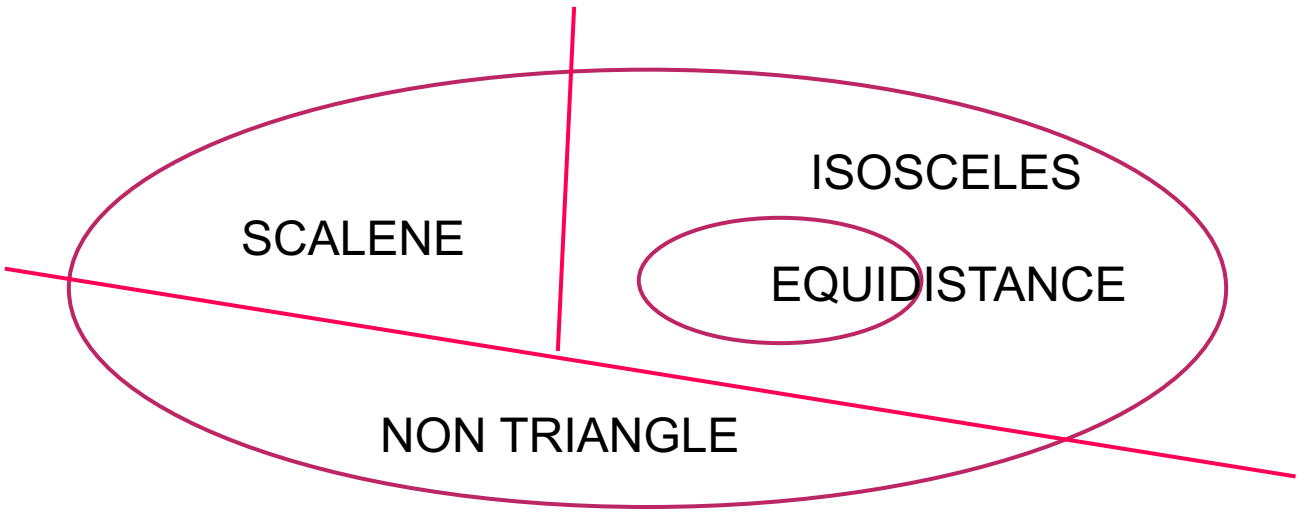
TryType triangle(Float a, Float b, Float c) { … }

If a, b, c represent the sides of a triangle, this methods determines the type of the triangle.

White box: test the function such that you cover every branch or every prime path in the code.

Let's now try to come up with test cases for this function from the **black box** perspective.

A possible partitioning triangle()'s input domain



input domain of triangle

This suggests 4 test-cases, one for every partition.

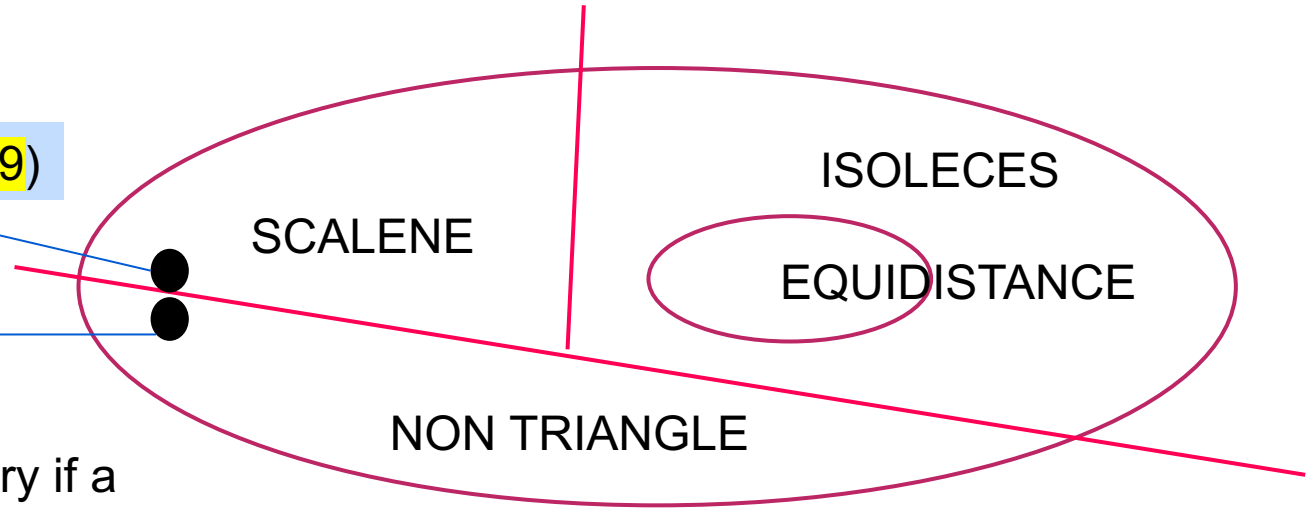
Additionally, we can also do boundary value test

Errors often lurk in the "boundaries" between partitions → test values at the boundaries.

Examples:

triangle(1,2,2.999)

triangle(1,2,3)



A test is at boudary if a slight change to its value would make it to switch to a neighbouring partition.

Example 2

int **incomeTax**(int income, int age, int children)

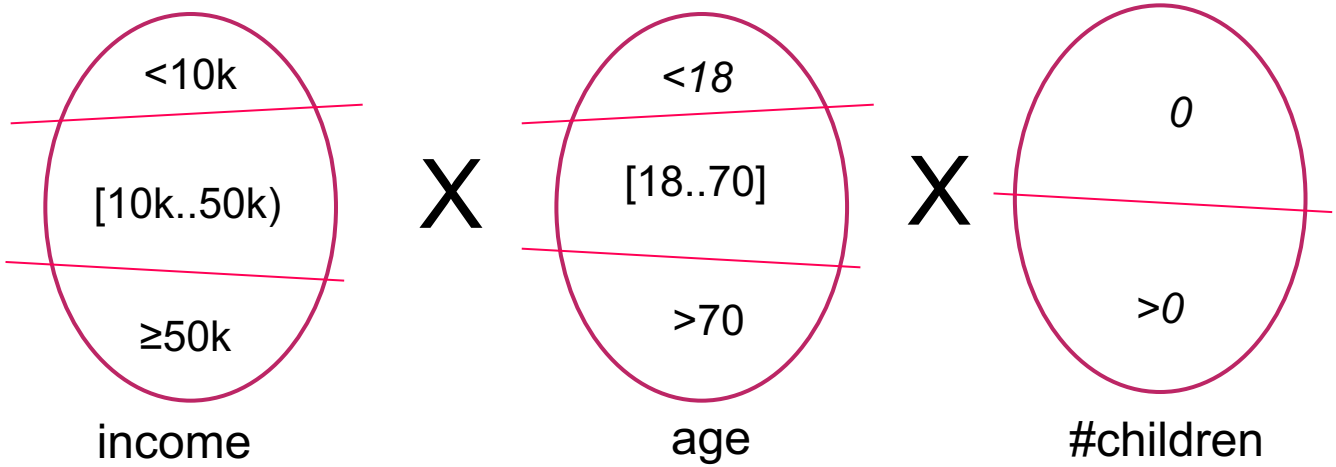
Fragments from its informal specification: *the method calculates income tax:*

- *Income below 10K is not taxed.*
- *A person under 18y or above 70y is not taxed, as long as the income is below 50K.*
- *Tax reduction applies, linear to the number of children the person has.*

This function has a more complex input space than the "triangle" example.

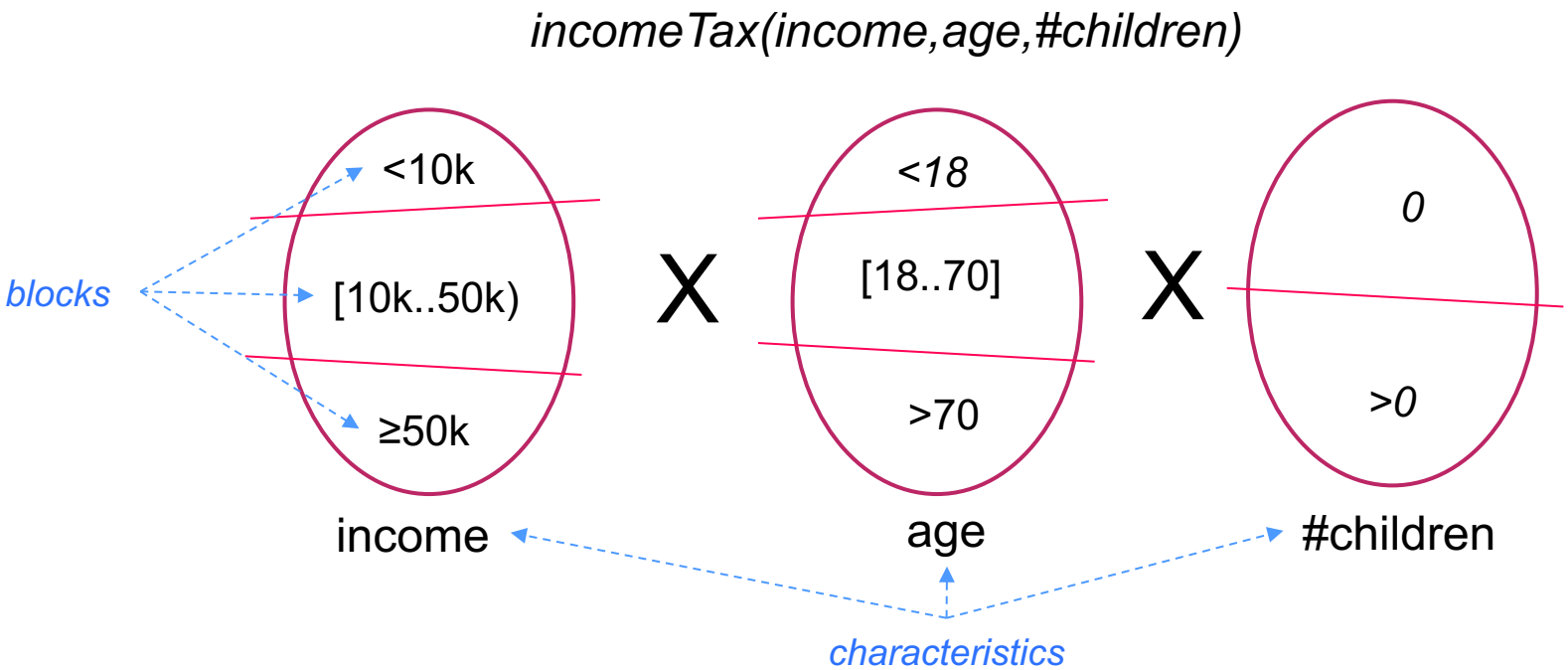
Consider the following partitioning

The input domain of *incomeTax(..)*



- As before, we can try to come up with test cases that would cover every partition.
- Note that just 3 test cases can cover all partitions!
  → does not feel very strong. This is because the approach ignores that different parameters may ”interact”. E.g. when the age is in [18..70], the tax depends on the number of children.

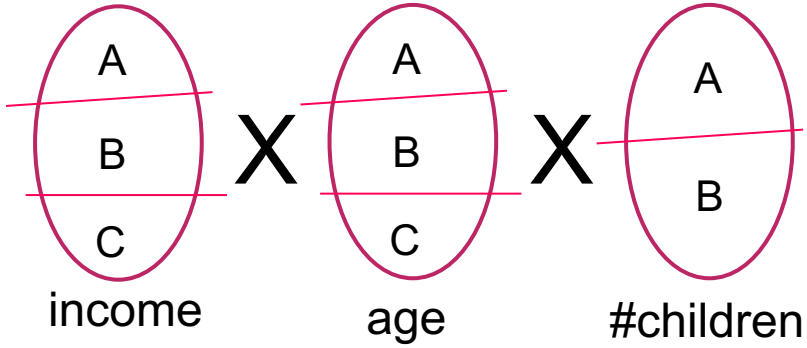
Combinatoric testing



- In this example the input domain is spanned by 3 characteristics, and there are in total 8 blocks.
- A block combination over the characteristics, e.g. (<10k,<18,0) abstractly specifies a test case.
- There are in total 3x3x2 = 18 such combinations of blocks.

Combinatoric testing

Let's name the blocks for convenience:

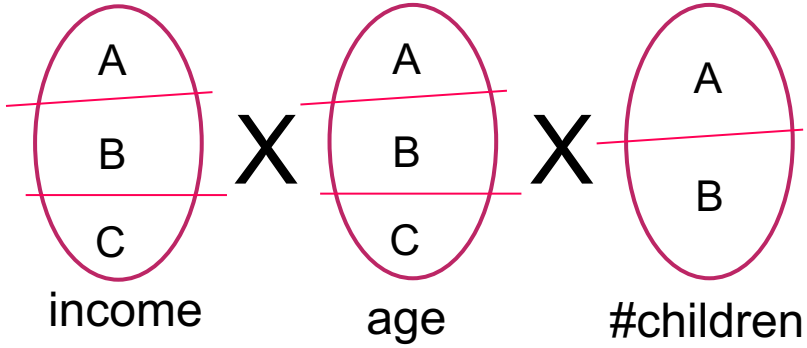


- Stats: 8 blocks, 18 combinations.
- (C4.24/2nd Ed. C6.2, **EACH CHOICE** coverage) Each block must be tested.
  $|T| = (\mathbf{max}\ i\text{: } 0 \le i < k\text{: } B_i)$; usually too weak.
- (C4.23/2nd Ed. C6.1, **ALL** coverage) All combinations must be tested.
  $|T| = (\Pi\ i\text{: } 0 \le i < k\text{: } B_i)$; does not scale up.

Completeness vs Scalability

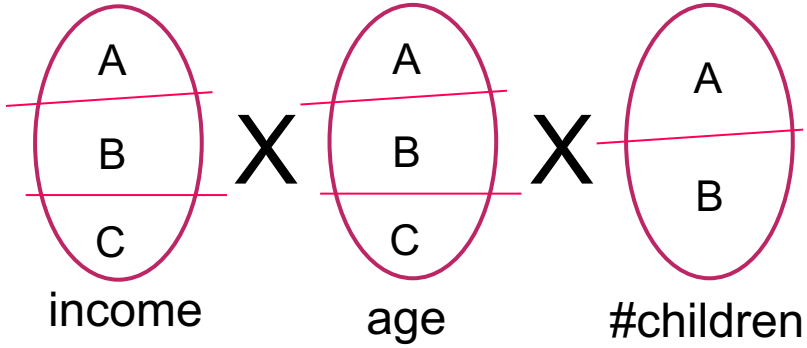
- The **ALL** coverage is complete, but not scalable.
- We'll discuss several "middle-way" coverage criteria, that are more practical.
  But keep in mind, by being less complete we may miss bugs.
- The actual effectiveness of a chosen coverage criterion depends on its alignment with the actual programming patterns and bug patterns of develepers. Unfortunately, these are not quantifiable.

How about to just cover all pairs?



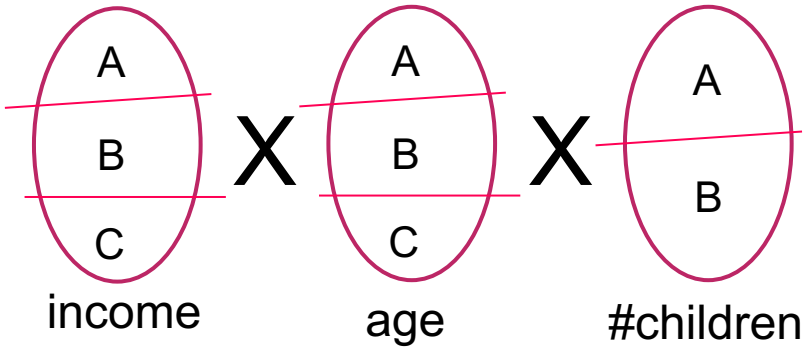
- Premise: most of the times, only *t* out of *N* parameters actually influence the behavior. E.g:
  - When the income is ≥50k, there will be tax. #children matters, but the age does not matter.
  - When the income is less than 10k, there will be no tax, regardless the age and #children.
- If we speculate on *t*=2, this leads to "pair-wise" testing: to cover all block-pairs over different characteristics.

Pair-wise and t-wise testing



- (C4.25/2nd Ed. C6.3, **pair-wise coverage**). Each pair of blocks (from different characteristics) must be tested.
- There are 9+6+6 = 21 pairs to cover. But we can cover them with just 9 test cases.
- Pair-wise coverage is stronger than EACH CHOICE, and still scalable.
- (C4.26/2nd Ed. C6.4, **t-wise coverage**). Generalization of pair-wise.

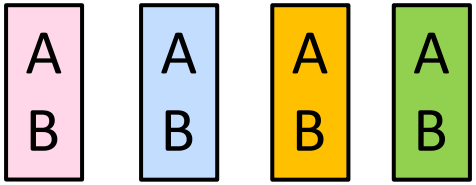
Example: a test set with full pair-wise coverage



	income	age	#children
Test-case 1	A	A	A
Test-case 2	A	B	B
Test-case 3	A	C	A
Test-case 4	B	A	B
Test-case 5	B	B	A
Test-case 6	B	C	B
Test-case 7	C	A	A
Test-case 8	C	B	B
Test-case 9	C	C	Does’t matter

PWC, example 2

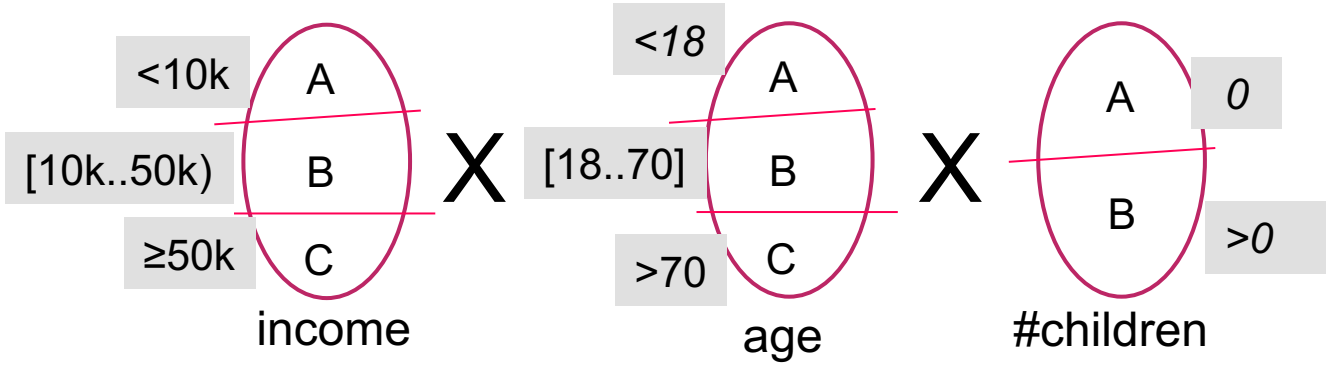
- Four characteristics, each with two blocks:



- Minimal test set that gives PWC:

	cha1	cha2	cha3	cha4
tc1	A	A	A	B
tc2	B	B	A	A
tc3	A	B	B	B
tc4	B	A	B	B
tc5	A	A	B	A

Consider again this test set



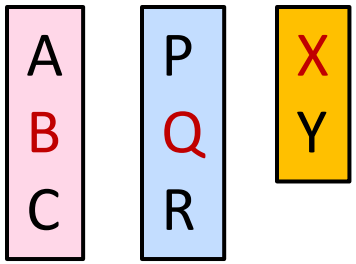
	income	age	#children
tc1	A	A	A
tc2	A	B	B
tc3	A	C	A
tc4	B	A	B
tc5	B	B	A
tc6	B	C	B
tc7	C	A	A
tc8	C	B	B
tc9	C	C	A

- Despite giving full pwc these tests arguably miss some important cases:
  - tax reduction on mid ages: (B,B,B)
  - tax reduction on young parent: (C,A,B)
- Solution: add constraints, but... (see next slide).

Pair-wise and t-wise testing

- Consider a program P with *N* characteristics [1..*N*]. For simplicity, suppose **each** characteristic is split into *B* number of blocks.
- Total number of pairs = $B^2 * V_2$ where V_2 is the number of subsets of size 2 out of *N* characteristics. So, $V_2 = \binom{N}{2} = \frac{N!}{2! * (N-2)!}$
- To cover all pairs you need at least B^2 number of test cases. Expect that it can be more (as in Example 2).
- In general, finding a minimum size test set that gives full t-wise coverage is **not** trivial.
- However, as pointed out before, k-wise testing ignores the "semantic" (that some combinations should be included or excluded (because they are not sensical)). We can add constraints, though this makes the problem of calculating the minimum test set even harder.

Adding a bit of semantic



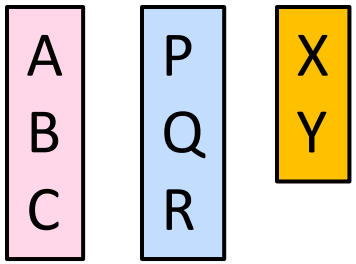
Example: t_0 = **(B,Q,X)**, generates these additional test requirements :

(B,Q,Y)
(A,Q,X) (C,Q,X)
(B,P,X) (B,R,X)

(C4.27/2nd Ed. C6.5, **Base Choice Coverage**, BCC) Decide a single *base test* t_0. Make more tests by each time removing one block from t0, and forming combinations with all remaining blocks (of the same characteristics).

$$|T| = 1 + (\Sigma i : 0 \leq i < k : B_i - 1)$$

Consider again



Base test t_0 = (**B,Q,X**), and these additional test, giving full BCC:

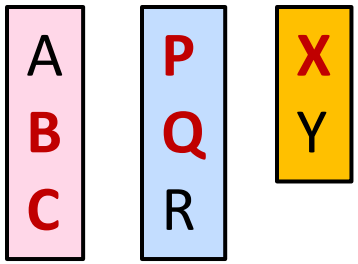
(B,Q,Y)

(A,Q,X) (C,Q,X)

(B,P,X) (B,R,X)

- What if we also need to insist on testing out all combinations of (C,P,-) ?
- Proposal: use multiple “base tests”

MBCC



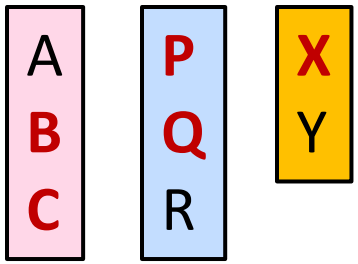
Decide the "base blocks": (red)
Choose one or more base tests. They can only use base blocks. E.g.:

base test t_0 = **(B,Q,X)**
base test t_1 = **(C,P,X)**

(C4.28/2nd Ed. C6.6, **Multiple Base Choices** coverage). For each characteristic we decide at least one *base block*. Then decide a *set* of base tests; each only include base blocks. For each base test, generate more tests by each time removing one base block, and forming combinations with *remaining non-base* blocks.

$$|T| \text{ at most } M + M^*(\Sigma i : 0 \leq i < k : B_i - m_i)$$

Example MBCC



The base blocks are marked red. The base tests:

base test t_0 = **(B,Q,X)**

base test t_1 = **(C,P,X)**

We need to add these tests to get full MBCC:

- Varying t_0 over non-base blocks:
  (A,Q,X), (B,R,X), (B,Q,Y)
- Varying t_1 over non-base blocks:
  (A,P,X), (C,R,X), (C,P,Y)
- Remove duplicates (there are none above)

Example-2, MBCC

A	**P**	**X**
B	Q	**Y**
C	R	Z

Red *: base blocks*

Chosen base tests = **(A,P,X)**, **(A,P,Y)**

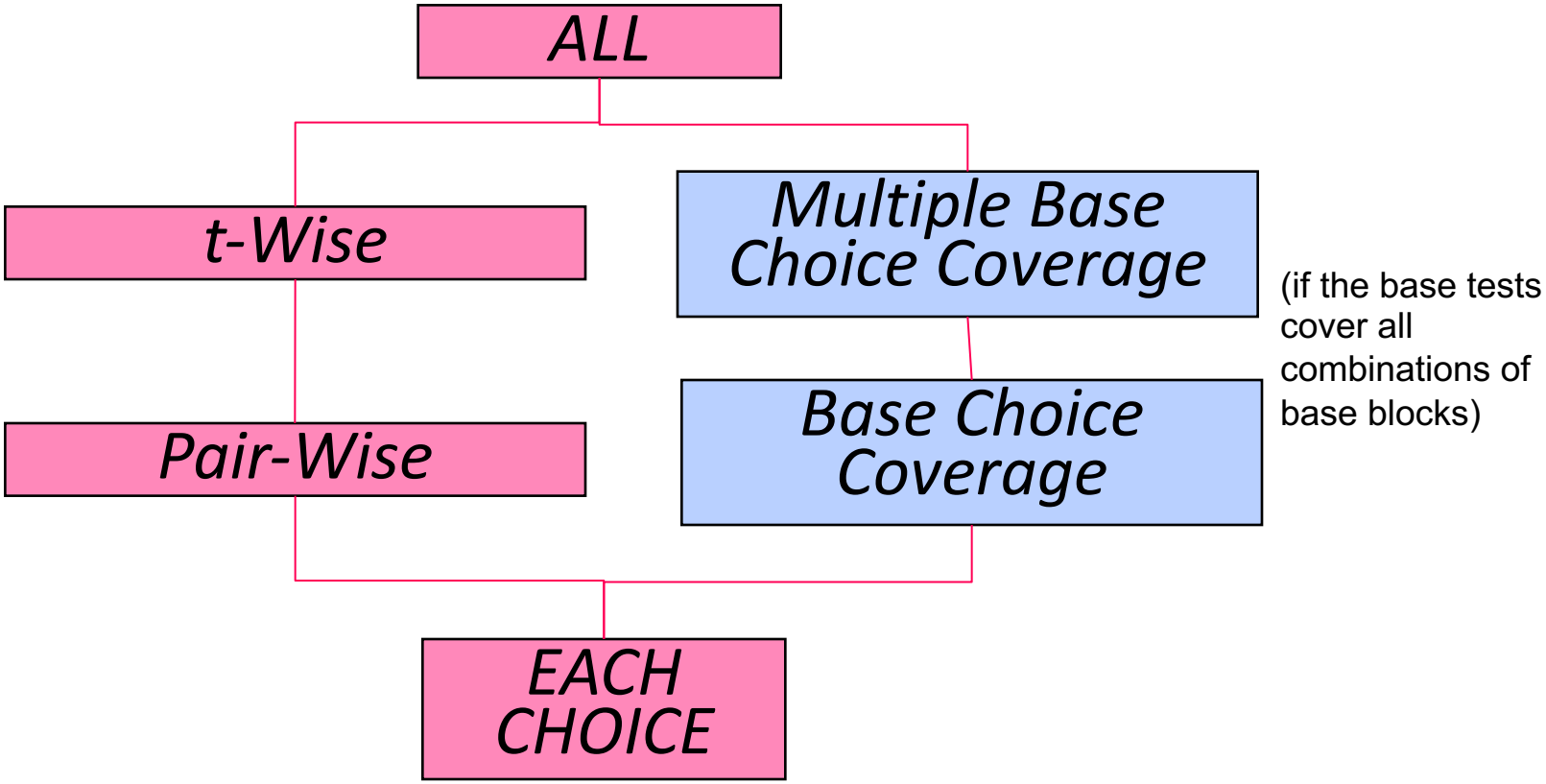
These produce these additional test requirements:

(B,**P,X**)	(**A**,Q,**X**)	(**A,P,**Z)
(C,**P,X**)	(**A**,R,**X**)	
		(**A,P,**Z) duplicate
(B,**P,Y**)	(**A**,Q,**Y**)	
(C,**P,Y**)	(**A**,R,**Y**)	

Some properties to note:

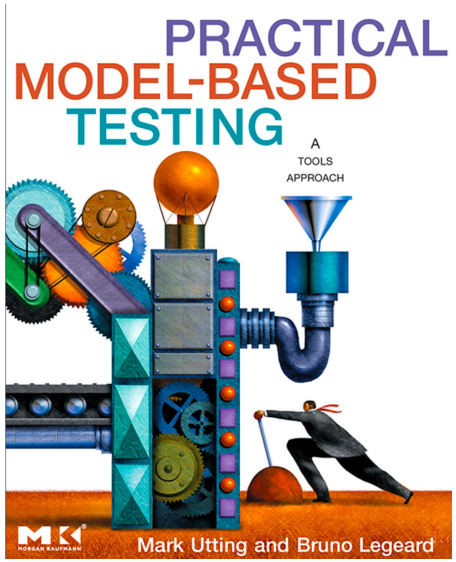
- base-blocks are not cross-combined except as in the base tests.
- non-base blocks are not cross-combined with each other.
- BCC and MBCC will also cover every pair of (base-block,non-base-block).

Overview of partition-based coverage



Model-based Testing

not covered in A&O



Model-based Testing (MBT)

- We use a model, e.g. a finite state machine (FSM), to abstractly describe possible observable behavior of a system under test.
  - it helps us in understanding the system.
  - It can be used as a specification that defines the correctness of the system.
  - It provides guidance on how to systematically test the system

Model-based Testing (MBT)

- Usually a black-box approach. We have limited visibility (we can't directly inspect the system states).
- We'll focus on the concepts rather than practice
- Limiting ourselves to deterministic systems

MBT Plan

We'll dicuss three setups with different level of observability :

- Can only observe accept/reject
- System emits output (so we can observe it)
- Can, to some degree, inspect the state

Simple accept/reject setup

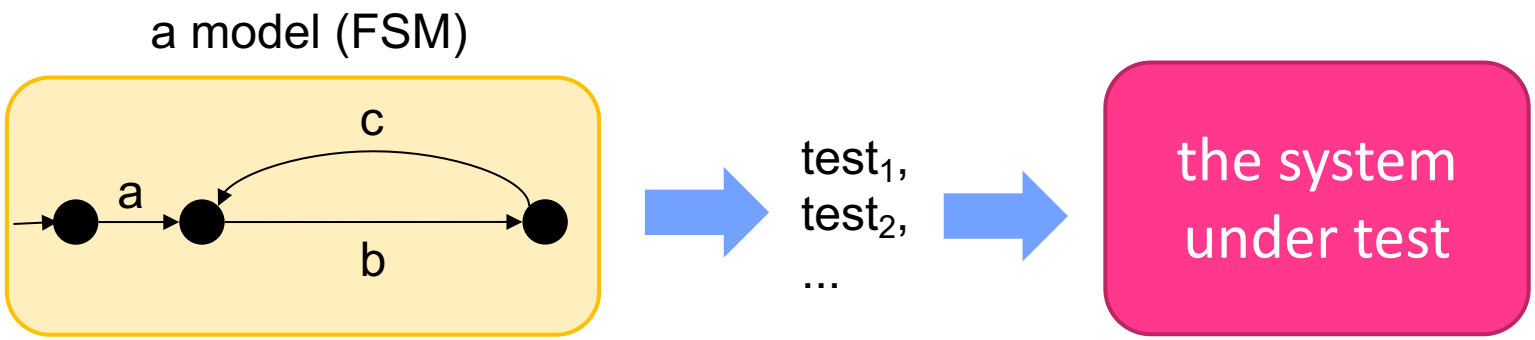
a control panel

- Imagine a system under test S that can be interacted with a set of actions e.g. { button-a, button-b, button-c }.
- **Observability assumption**:
  - If we do an action on S, we can observe either S accepts the action, or it rejects it (e.g. displaying an error-msg, or by crashing).
  - we can't (or it is difficult to do it) peek into S state.
- We limit ourselves to **deterministic** S: when the system is *at the same state*, it will *always do the same thing* (e.g. it does not "sometimes" accepts a, and other times not)

Observable behavior of S

- An (observable) behavior is a sequence of actions that S accepts:
  - ε (empty sequence of actions) is a **behavior** of S
  - if σ is a behavior of S, and if after that it accepts action a, then σa is a **behavior** of S.
- Define **obehv**(S) = set of all possible (observable) behaviors of S.
- Note that if $\sigma \notin$ **obehv**(S), σ has a prefix υa, where the sequence υ is still a behavior of S, but after that it rejects a.

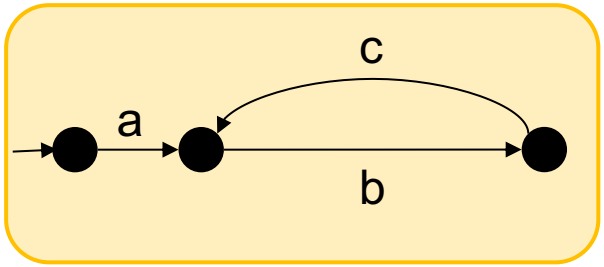
Finite State Machine (FSM) model



- An **FSM** models the behavior of a system under test. Such a model is essentially a tuple (Σ,s_0,A,T) where:
  - Σ is a (finite) set of states (the nodes in the above example). One, s_0, is marked as the initial state.
  - A is a set of actions.
  - T is a set of transitions (arrows) between states. Each arrow $s \xrightarrow{a} t$ is labbeled by the action a that triggers the transition.
  - In the above example the FSM has three states (the nodes), three transitions, and A = {a,b,c}.

Behavior of an FSM

an FSM M :

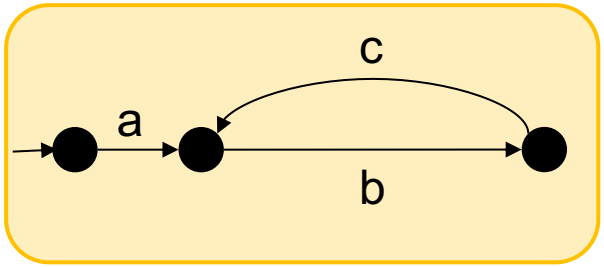


- Let M be an FSM. A finite sequence of actions is a **behavior** of M if it can be obtained from a path in M that starts in M's initial state.

  Example: *ab* is an execution of the above M, but *ba* is **not** an execution of M.

- Define: **behv**(M) = the set of all possible behaviors of M.

Model-based Testing of S

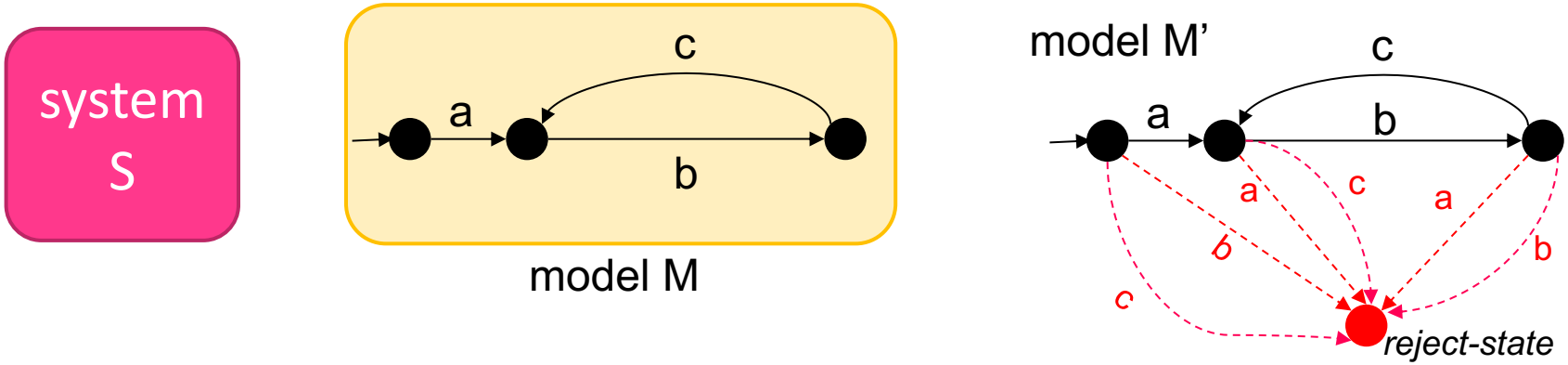


- Let S be a system. S correctly implements a model M iff (if and only if) **obehv**(S) = **behv**(M).
- This definition gives you a way to derive tests from the model M. For example, we could have these tests:

t_1: σ = abc	expectation: $\sigma \in$ obehv(S)	(postive test)
t_2: σ = abcbc	expectation: $\sigma \in$ obehv(S)	(postive test)
t_3: σ = b	expectation: $\sigma \notin$ obehv(S)	(negative test)
t_4: σ = c	expectation: $\sigma \notin$ obehv(S)	(negative test)

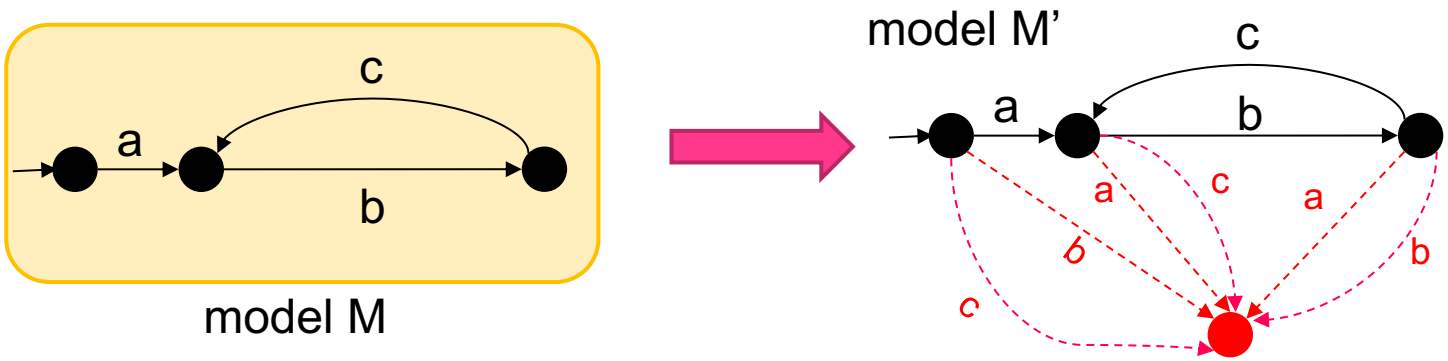
- We can't usually do exhaustive testing as the set of possible tests is usually infinite. So you need a coverage criterion.

Coverage Criterion



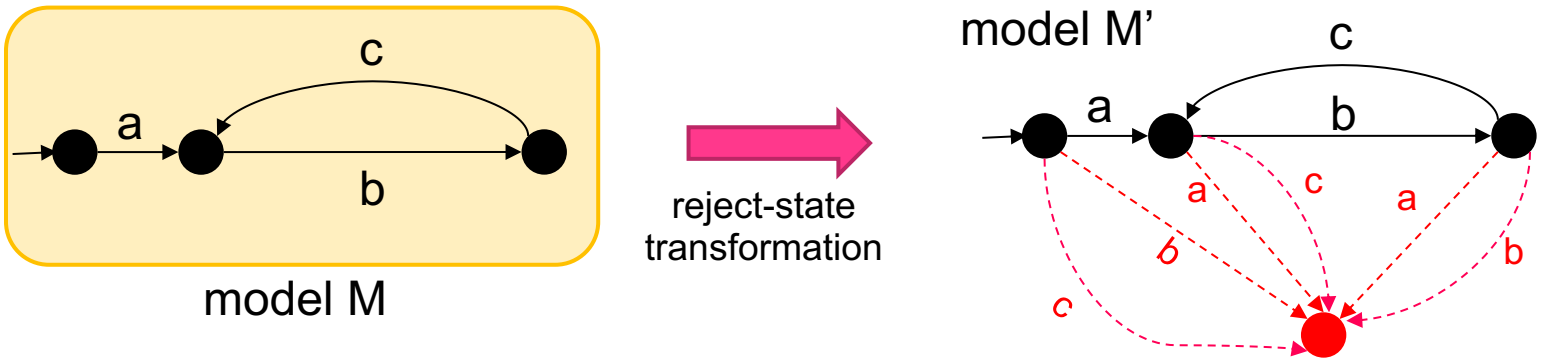
- Transition coverage: the test requirement (TR) is the set of all transitions in M (we have to cover these).
- Prime path coverage: TR = the set of all prime paths in M
- **However,** this this will leave out negative tests (those that should be rejected).
- **Reject-state Tranformation:** convert M to M' as in the example above, by adding "rejection" as a special sink-state.
- **Positive test** is a $\sigma \in$ **behv**(M) (so, it does not end in the reject state)
- **Negative test** is a $\sigma \in$ **behv**(M') that ends in the reject state. Note that such σ is thus $\notin$ behv(M).

"reject-state" transformation



- Formally the reject-state transformation is defined as follows. Given an FSM M = (Σ,s_0,A,T), we transform it to M' = (Σ' ,s_0, A, T') where:
  - $\Sigma' = \Sigma \cup \{$ reject $\}$, where reject is a special state assumed not a member of Σ.
  - $T' = T \ \{ s \xrightarrow{a}$ reject $\mid \ s \in \Sigma ,\ a \in A ,\ (\forall\, t : t \in \Sigma : s \xrightarrow{a} t \notin T) \ \}$
    So, T' is obtained by adding ougoing transitions from every state in the original M, that would be disallowed by the original M.

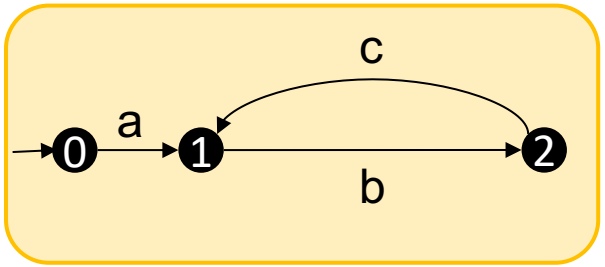
Coverage Criterion



- **+/- model transition coverage**. TR: we should have both postive and negative tests, such that :
  1. The positive tests give full transition coverage over M
  2. The negative tests give full transition coverage over M'

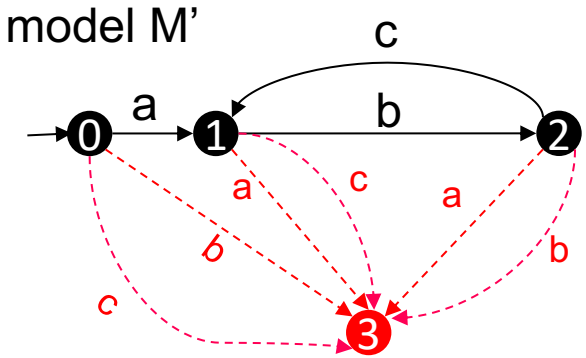
- Similarly we define **+/- model prime path coverage**.

Coverage Criterion



model M

reject-state transformation



prime paths of M and M' :

paths	M	M'
012	✓	
121, 212	✓	✓
0b3, 0c3, 01a3, 01c3, 012a3, 012b3		✓
21a3, 21c3		✓

- Example test set with full **+/- model transition coverage** :
  Positive tests : { abc } Negative tests : { b , c , aa, ac, aba, abcbb }
- Example test set with full **+/- model prime path coverage** :
  Positive tests : { abcb }
  Negative tests : { b , c , aa, ac, aba, abb, abca, abcc, abcba }

System with input/output

a vending machine

- Imagine a system S that have input actions (user actions) and ouput actions (e.g. dispense-cola). Tester can control input actions, but not output actions.
- Assumed observability:
  - we can observe if S accepts/rejects an input action.
  - we can observe output actions.
  - we can observe if S gives no output, when it is supposed to give an output action (e.g. by waiting long enough)
  - we can't peek into S state

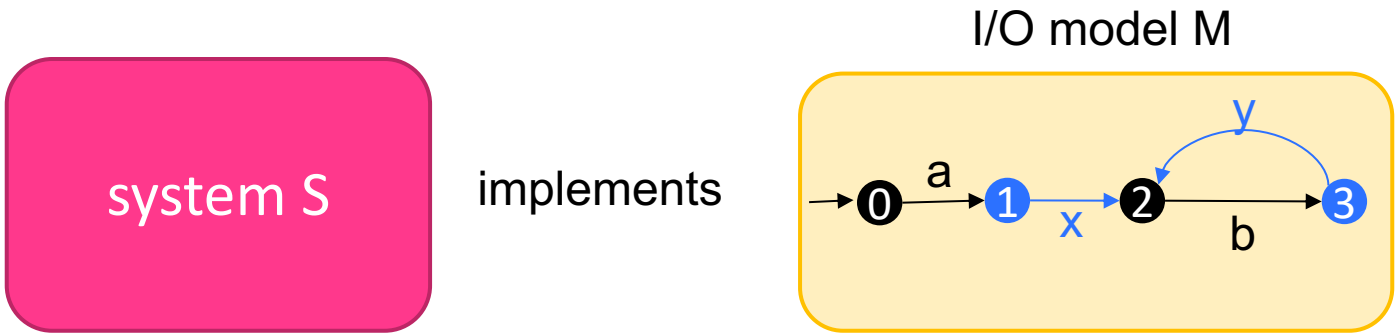
System with input/output

- We'll again limit ourselves to deterministic S → always behave the same way when it is at the same state.
- Also this simplifying assumption: when at a state the system can accept an input, it won't produce output while at the same state.

System with input/output

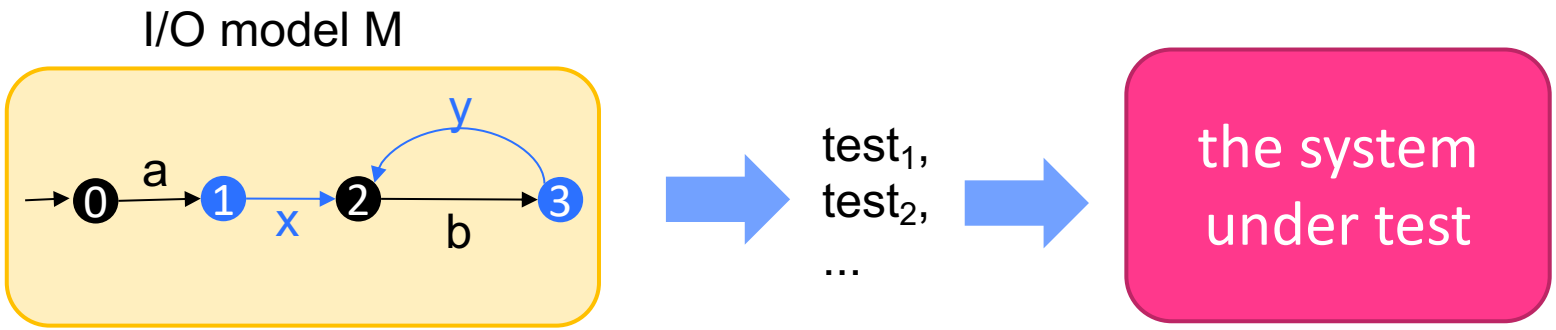
- Let I be the set of S input actions, and O its set of output actions.
- Def. observable behavior of S:
  - ϵ is a **behavior** of S
  - if σ is a behavior of S, and if after that it accepts an input action a∈I, then σa is a **behavior** of S.
  - if σ is a behavior of S, and if after that it does output action x∈O, then σx is a **behavior** of S.
- As before, **obehv**(S) = the set of all observable behaviors of S.

FSM with I/O



- An **FSM M with I/O** is tuple (Σ,s_0,A,O,T) where:
  - Σ, s_0, A, and T mean as before.
  - O ⊆ A is the set of output actions. Define I = A/O: the set of input actions
  - For every state, all outgoing transitions are all labelled by input actions, or all labelled by output actions.
- A state s is an **input state** iff its outgoing transtions are labelled with input actions.
- We'll limit ourselves to deterministic M : every behavior σ∈**behv**(M) should lead to a unique state.
- As an example, for the above M, I = {a,b} and O= {x,y}

FSM with I/O



- As before, S correctly implements M iff o**behv**(S) = **behv**(M).
- Example of tests:

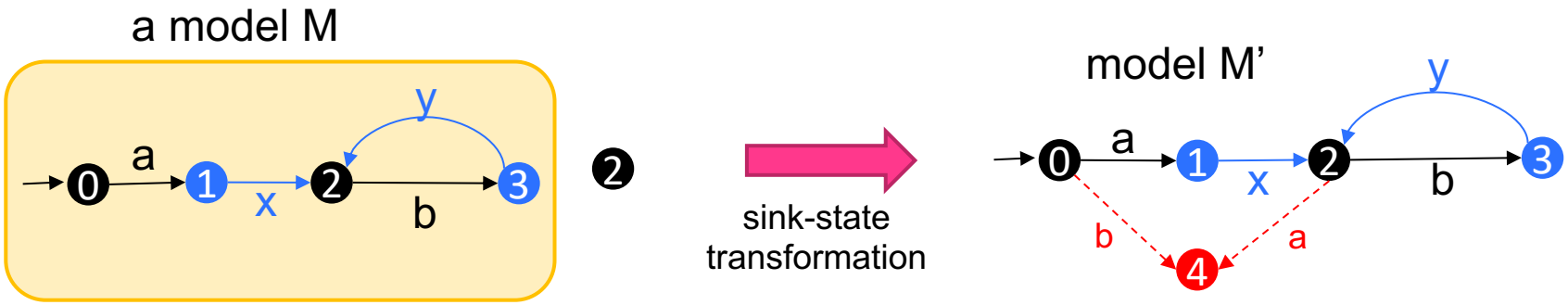
  t_1: σ = ax expectation: a∈obehv(S), then S outputs x (postive test)

  t_2: σ = axby expectation: axb∈obehv(S), then S outputs y (postive test)

  t_3: σ = b expectation: σ∉obehv(S) (negative test)

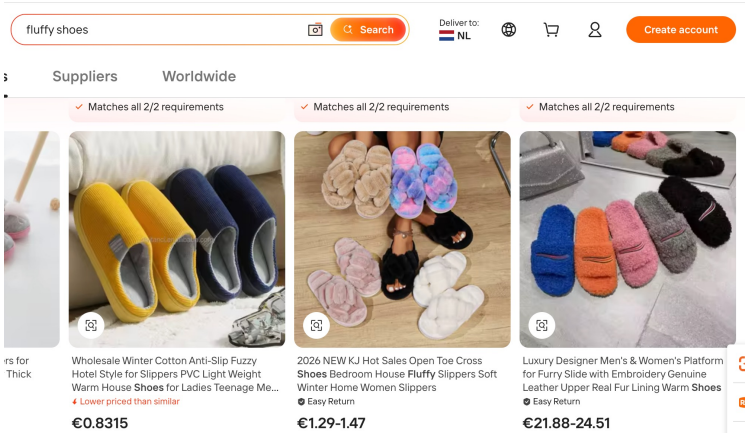
  t_4: σ = axa expectation: σ∉obehv(S) (negative test)
- We can define **+/- model prime path coverage** as before, but the used sink-state transformation should be adjusted a bit.

FSM with I/O



- Sink-state transformation should only add transitions from input states of M, and should only range of input actions.
- A test set that gives full **+/- model transition coverage**:
  - positive tests: { axby }
  - negative tests: { b, axa, axbya }
- A test set that gives full **+/- model prime path coverage**:
  - positive tests: { axbyby }
  - negative tests: { b, axa, axbybya }

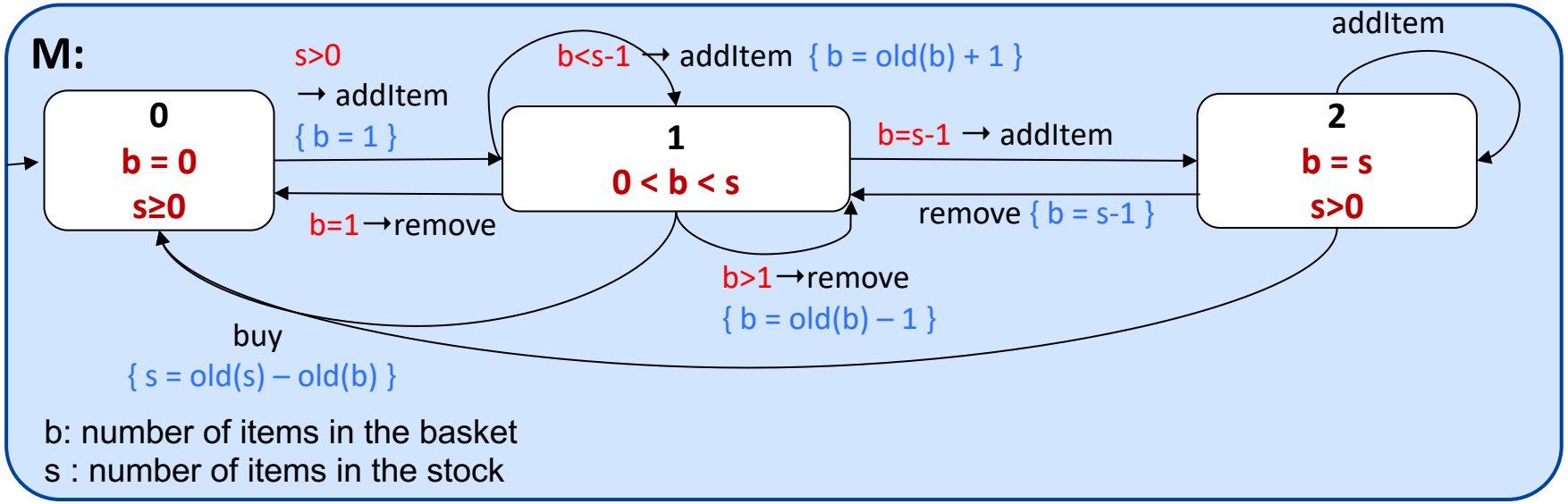
System with limited access to its state



a control panel

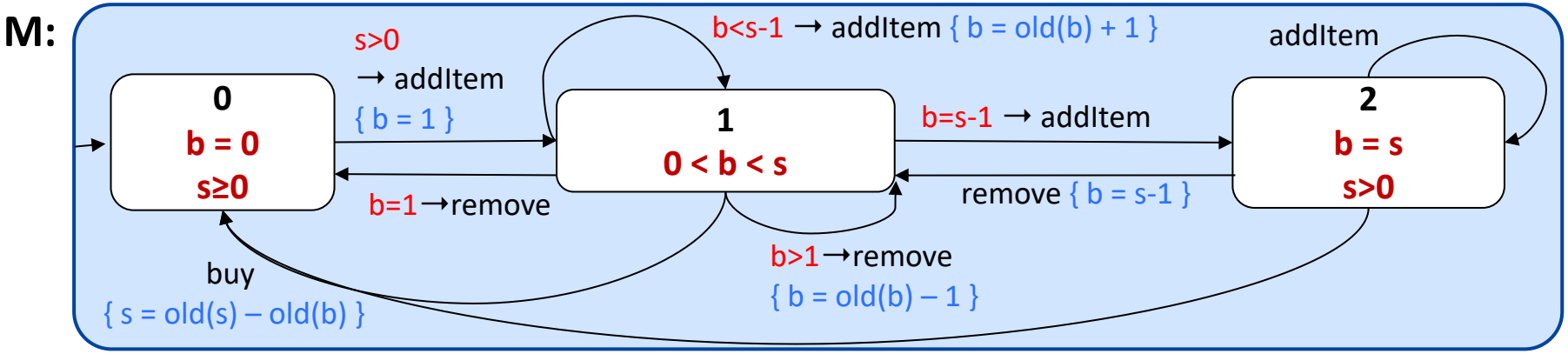
- Imagine a system S that has input actions (user actions).
- Observability: we can inspect some selected variables in S state

Extended FSM (EFSM)



- FSM extended with variables. Above: {b,s}. Very expressive, as the underlying state space can be infinite.
- States have predicates that are expected to hold.
- Transitions may be guarded and may have post-conditions: g → a {Q} : the transition is triggered by the action a, but it can only happen if the guard g is true. If it happens, the transition is expected to establish the post-condition Q.
- Expression "old(x)" in the post-condition of a transition: the value of variable x at the start of the transition

Deterministic EFSM

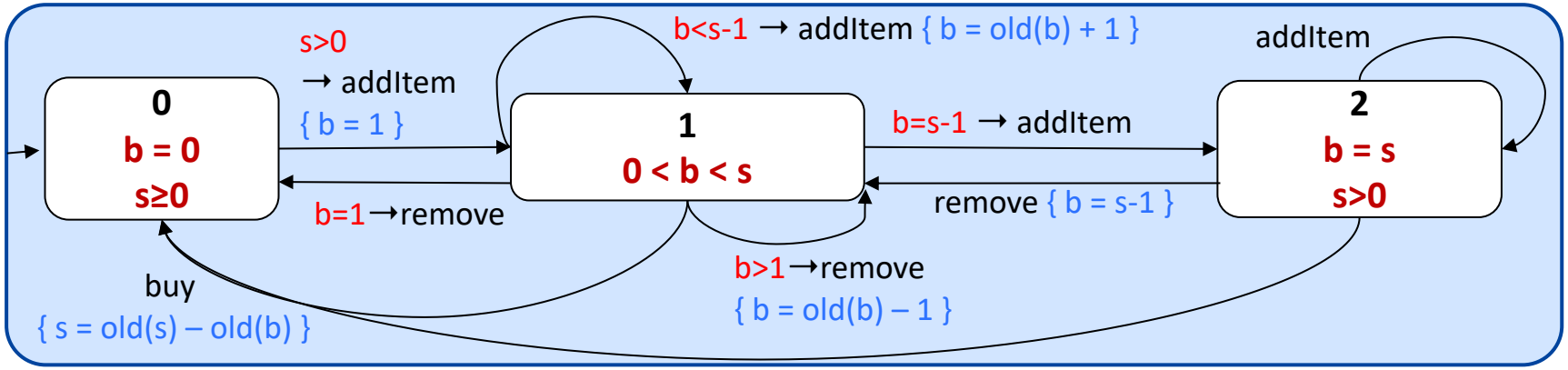


- We'll restrict to **deterministic** EFSM:

  Any two transitions s $\xrightarrow{g \to a\ \{Q\}}$ t and s $\xrightarrow{h \to a\ \{R\}}$ u that go out from the same state s and labelled by the same action a, should not have overlaping guards (so, the conjunction g∧h should be unsatisfiable).

Run of an EFSM

M:

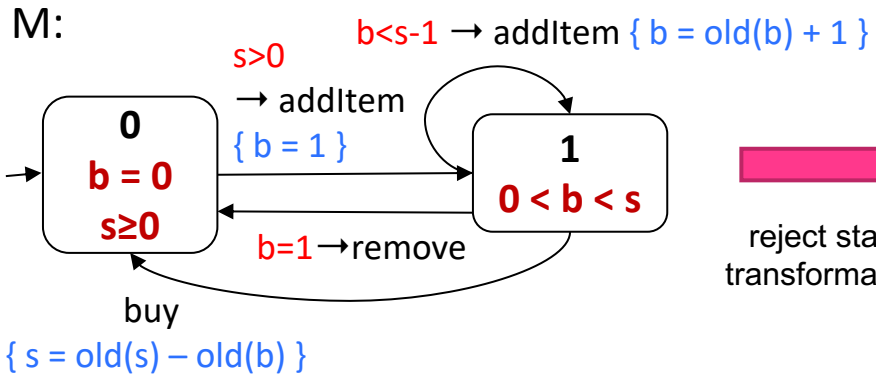


runs

- A finite run of an EFSM M is a finite sequence of transitions of in M that starts in its initial state. More precisely:
- s_0 is a run of M, where s_0 is M's initial state.
- If σ is a run of M that ends in state s, and $s \xrightarrow{l} t$ is a transition in M, then $\sigma \xrightarrow{l} t$ is a run of M.
- Define **runs**(M) = the set of all possible runs of M. Every run of M is a positive test.

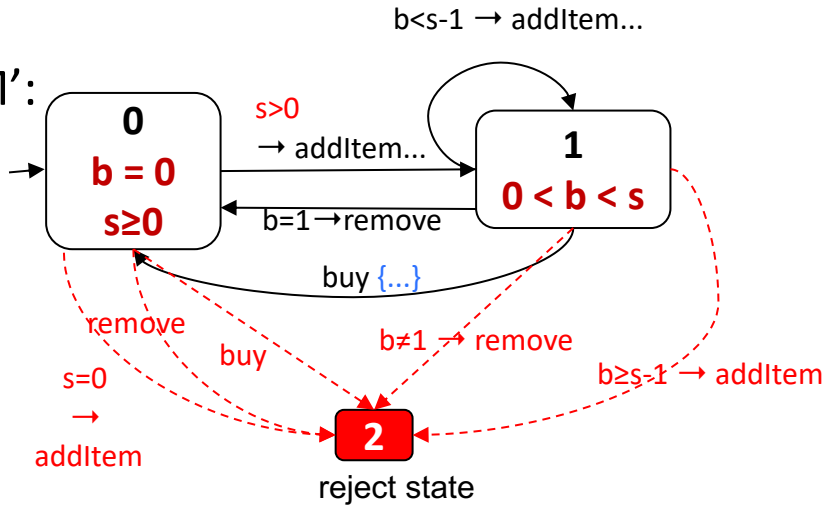
Negative tests

Suppose this is M:



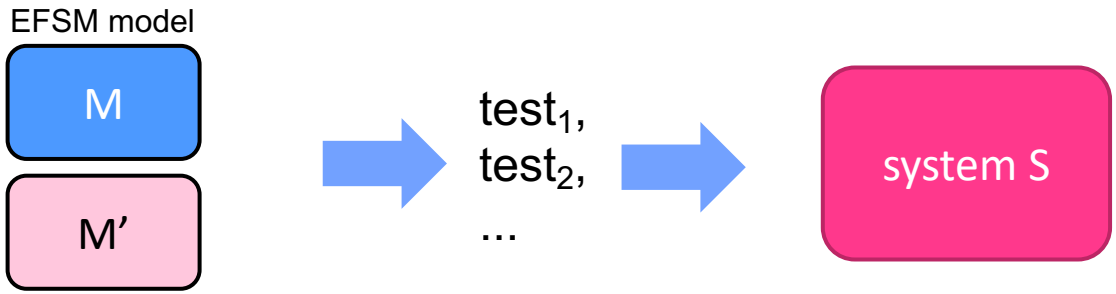
reject state transformation

M':



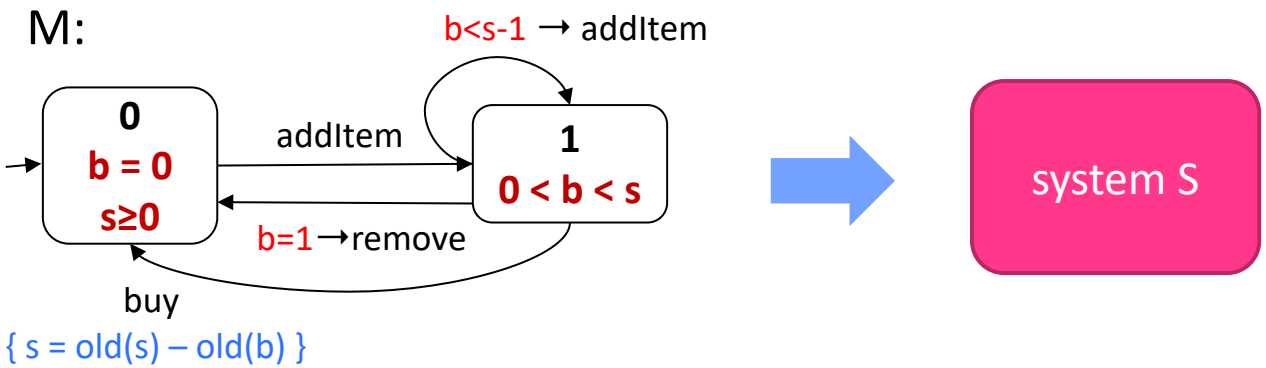
- Reject-state transformation adds transitions that would have been disallowed by the original M.
- We define a negative test as a run in M' that ends in the reject state.
- We can now come up with tests that satisy e.g. the **+/- model transition coverage** or the **+/- model prime path coverage** (the definition of these criteria remain the same).

Using tests



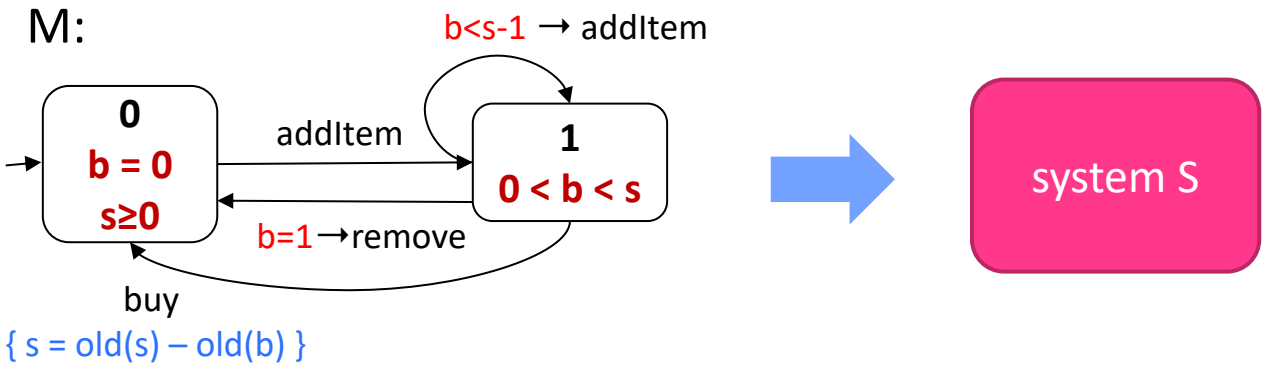
- A run of the model M can be used as a positive test to test S. Similarly, using M' we can obtain negative tests.
- Given a positive test σ, we expect that S should be able to do all the actions prescribed in σ and moreover S states passed during the test should satisfy all the transitions' post-conditions and state-predicates in σ.
- Let first define some supporting concepts.
- Given a model run σ, S actual run (concrete run) when given σ as a test is a sequence of S concrete states, denoted by **crun**(σ). Defined as follows:

Using tests



- Concrete run of S:
  - **crun**(s_0) = z_0, where s_0 is M's initial state and z_0 is S actual initial state.
  - Let $\rho = \sigma \xrightarrow{g \to a\ \{Q\}} t$ be a run of M. If **crun**(σ) < $|\sigma|$+1, it means that S can't do the full σ. We define **crun**(ρ) = **crun**(σ). $|\sigma|$ is defined as the number of transitions in σ.
    Else, and let s be the last state of σ. Let z be the last state of **crun**(σ). If g holds on z and S can do the action a at the state z, resulting in a new state z', then **crun**(ρ) = **crun**(σ) ++ [z'].
    Else **crun**(ρ) = **crun**(σ).
- For example if σ is $0 \xrightarrow{addItem} 1$, **crun**($\sigma$) could be [<b=0,s=100>, <b=1,s=100>]

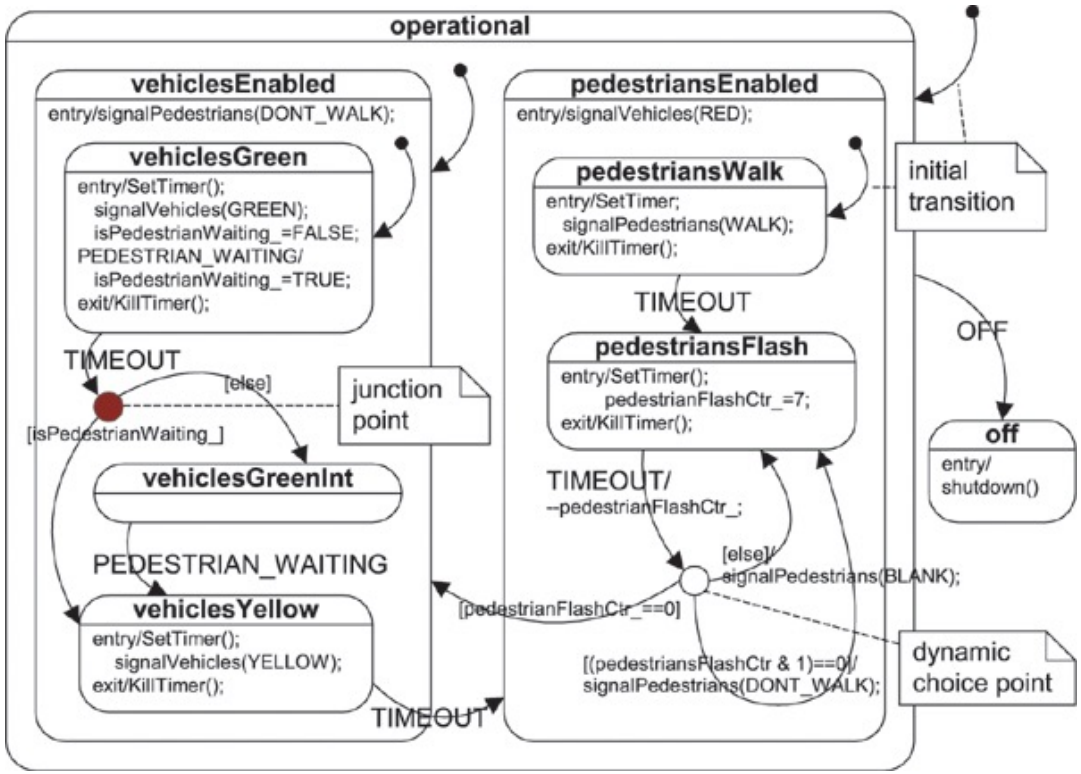
Using tests



- Define: S **accepts** σ if $|\text{crun}(\sigma)| = |\sigma|+1$
- Define: S is correct along σ if
  1. **pred**(s_0) holds on z_0, where **pred**(s) is the conjunction of all predicates that decorate the state s in M.
  2. for all $0 \le k < |\sigma|$, let ... $\xrightarrow{g \to a\ \{Q\}}$ t be the k-th transition in σ, and let z be the k+1-th state in **crun**(σ); we have Q and **pred**(t) hold on z.

- S passes a positive test σ if S accepts σ and S is correct along σ.
- S passes a negative test σ if S does not accept σ.

Hierarchical EFSM in UML



There are various modelling tools in the market, typically offer more extensive modelling language that what we discussed. E.g. UML includes “hierarchical state machines”, which are essentially EFSM + hierarchy to provide more organization when the models become complex.